

Perfect  — I've compared both files in detail:

1. Common Foundation (Same in both)

Both `psss_research.py` and `adaptive_vulnerability_prioritization_with_mitre_att&ck_integration.py` share the **same full CVSS → EPSS → ATT&CK pipeline**:

Stage	Purpose	Notes
CVSS Parsing	Extracts base metrics (AV, AC, etc.) from NVD JSON	Uses Logistic Regression models for text-based metric prediction
CVSS Base Scoring	Implements official CVSS v3.1 formula	Converts predicted metrics into base score
EPSS Integration	Merges exploit likelihood scores from Cyentia EPSS	Aligns on <code>cve_id</code>
PSSS (Static)	Combines CVSS and EPSS: $0.5*CVSS + 0.5*EPSS$	Static fixed weights
ATT&CK Mapping	Adds MITRE ATT&CK techniques via external CSV	Adds binary <code>attack_criticality</code>
Final PSSS	Combines all three: $0.4*CVSS + 0.4*EPSS + 0.2*ATT\&CK$	Still static weights
Visualization	Multiple <code>matplotlib</code> plots for before/after comparison	Scatter & bar plots

So far, both files are **identical up to this point**.

2. Key Difference — Adaptive Weighting Section

`psss_research.py`

- Has a **basic adaptive weighting** using logistic regression.

- Defines a **single threshold (EPSS > 0.5)** to label “exploited in the wild”.

Learns coefficients (α , β , γ) from logistic regression:

```
logreg_model.fit(X, y)
weights = logreg_model.named_steps["logisticregression"].coef_[0]
alpha, beta, gamma = weights / (abs(weights).sum() + 1e-8)
```

-

Outputs one adaptive column:

```
df_attack["PSSS_adaptive"]
```

-
- ⚠ If the dataset is imbalanced (e.g., all $epss < 0.5$), it **may fail or produce unstable weights**.



adaptive_vulnerability_prioritization_with_mitre_att&ck_integration.py

This is a **more robust and production-ready version** of the above.

✅ **Enhancements added:**

Robust handling of sparse ATT&CK coverage

```
if df_attack["attack_criticality"].sum() == 0:
    df_attack["attack_criticality"] = (df_attack["epss"].fillna(0) > 0.3).astype(int)
```

1. → Prevents crashes when ATT&CK data is missing.

2. Dynamic thresholds for “exploited” labels

- Runs for **two thresholds**: 0.5 and 0.7 .
- Produces **two adaptive scores**:
 - $PSSS_adaptive_thr0.5$

■ PSSS_adaptive_thr0.7

Automatic fallback to static weights

```
if df["exploited_in_wild"].nunique() < 2:  
    alpha, beta, gamma = 0.4, 0.4, 0.2
```

3. → Ensures model doesn't break on small samples.
4. **Debug visibility + better logging**
 - Prints learned weights clearly.
 - Displays column summaries for confirmation.
5. **Keeps baseline static version (PSSS_static)** for comparison.

Summary Table

Feature	psss_research.py	adaptive_vulnerability_prioritization_with_mitre_att&ck_integration.py
CVSS → EPSS → ATT&CK pipeline	✓ Yes	✓ Yes
Static weighting	✓	✓
Adaptive weighting	✓ (basic)	✓ (robust)
Handles missing ATT&CK	✗	✓
Multiple EPSS thresholds	✗	✓ (0.5 & 0.7)

Fallback to static weights	✗	✓
Debug/column logging	Minimal	Extensive
Usability	Prototype	Production-grade

🏁 Verdict



`adaptive_vulnerability_prioritization_with_mitre_att&ck_integration.py` is the **robust and improved** version — the one you should keep and document in your final report or demo.

It's adaptive, fault-tolerant, and supports multiple thresholds for better model calibration.

Would you like me to generate a **short research summary paragraph** explaining how the robust adaptive model improves reliability (for inclusion in your report/paper)?